Tank Wars Final Report

Paul Chap Anh Luong Jared Pringle Tong Wu

ABSTRACT

In this paper, we describe our video game implementation Tank Wars. The video game project was split into six major components: Processor, Memory Map I/O, VGA Controller, Game Code, NES Controllers, and Sound. Our baseline Instruction Set Architecture (ISA) for the processor is based on the National Semiconductor CR-16 micro-controller. Memory Map I/0 determines how to map instructions, data, and I/O devices to memory. Our VGA controller's primary objective is to display information on a standard VGA display. Game Code determines the behavior of the tanks as well as providing communication between I/O devices. Two NES controllers are used to provide input from each player in determining where a tank will move on-screen. There are four different sounds used for the game; a sound to begin the game, a sound to end the game, a sound effect for firing a bullet out of a tank, and a sound that simulates the movement of a tank.

What makes a video game project distinctive from other types of projects is the fact that video games provide a leisurely activity for others to enjoy. Users get to play an active role in their entertainment by interacting with a user interface via controller inputs while they get visual feedback from a video display. Not only can a video games be good fun, but they can also provide friendly competition between players. For our game, we have a two player simultaneous play video game where each player controls a tank and uses that tank to destroy an opposing tank.

The result of our project is the fact that we have created a competitive multi-player 2-D shooter video game that runs on the Spartan-3E FPGA Xilinx Board.

1. INTRODUCTION

The project we constructed is loosely based upon Battle City, the multi-directional shooter video game for the Nintendo Family Computer produced and published in 1985 by Namco. Tank Wars, our video game implementation, was coded using Verilog Hardware Description Language (HDL) and assembly language. Our video game runs on the Spartan-3E FPGA board; this powerful board contains 500,000 gate equivalents, 40Kbs of RAM, a 50 MHz crystal clock oscillator, 128Mbit Parallel Flash, 16 Mbit SPI Flash, 64 Mbyte DDR SDRAM, Ethernet connector, JTAG USB download connector, two 9-pin RS-232 serial ports, VGA output connector, PS/2-style mouse/keyboard port, rotary encoder with a push button, four slide switches, eight individual LED outputs, four momentary-contact push buttons, 100-Pin expansion ports, three 6-pin expansion connectors, and a 16 character-2 line LCD. The game features two player simultaneous play where the player, controlling a tank, attempts to destroy the opposing player who is controlling a second tank.

In order to work on this project as a group we decided to hold group conference meetings via Skype to discuss our implementation design ideas, code, schematics, issues that each member were encountering, and what days to meet up in person to put our project together. We used both the Digital System Lab in the Merrill Engineering Building and the Undergraduate Lab in Warnock Engineering Building Room 130 to do testing of our project. Various types of software programs were used either to assist in completing our project or to construct the project itself including Dropbox, ISE Project Navigator, Matlab, Microsoft Visual Studio, Notepad, Paint, PowerPoint, Skype, and Word. Our project was divided up into six major components which are the Processor, Memory Map I/O, VGA controller, Game Code, NES controllers, and Sound.

2. PROCESSOR

The Instruction Set Architecture (ISA) for our processor is based on the National Semiconductor CR-16 micro-controller. Our ISA is a subset of the CR-16 with simplified instruction encoding. The processor specification is based on RISC concepts and can be implemented as a two stage pipeline. Both word size and address space are 16 bits. Each address refers to a complete word, which is two bytes, so the address space is 2^(17) bytes. All instructions are single word. Most instructions refer to a 16 entry register file. Operation code is contained in the highest nibble, destination register address follows the next nibble, the remaining byte is used either as an immediate data value for some instructions or split into a four bit operation code extension and a four bit source register address for other instructions. The instruction set includes ANDI, ORI, XORI, ADDI, SUBI, CMPI, MOVI, MULI, LUI, AND, OR, XOR, ADD, SUB, CMP, MOV, LOAD, STOR, JAL, Jcond, LSHI, LSHI, LSH, EQ, NE, CS, CC, HI, LS, GT, LE, FS, FC, LO, HS, LT, GE, and UC. The processor is a 3 stage processing unit consisting of fetching, decoding, and executing. Write backs can take a cycle due to the possibility of a data hazard. Here is a schematic of the processor:

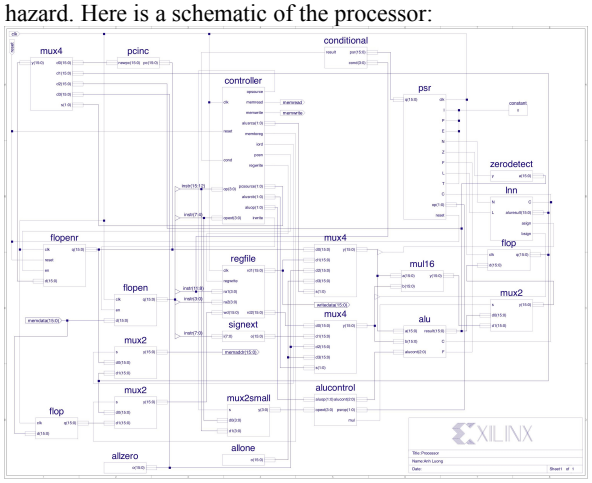

Figure 1. Processor

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Conference'10, Month 1–2, 2010, City, State, Country.
Copyright 2010 ACM 1-58113-000-0/00/0010…$10.00.

3. MEMORY MAP I/O

For storing game code and data we split memory into 16-bit data memory from 0 to 2048. Program instructions are mapped to 4000 addresses while data storage is mapped to 1000 addresses. Contained in data storage are information regarding the tanks such as if it is hit, what direction it is facing, the horizontal and vertical position of the tank, what direction its bullet is facing and the horizontal and vertical position of the bullet. Screen memory is mapped from c000 to d020. Although screen memory was declared from 0 to 8192, we did not utilize all of the memory. Screen memory takes up 4800 addresses in our implementation because we are using 8x8 pixels to represent a glyph and there are 80x60 locations to display glyphs on a 640x480 resolution display. A separate 32-bit memory block was used for glyphs. Glyphs are 24-bit addresses since we are utilizing 3-bit color for each pixel of the glyph. Each signal as well as I/O devices take up an address. Two NES Controllers took up 16 addresses; each controller having each respective button take up an address. Every sound we used for the game was also mapped to an address such as the game over, game start, bullet, and moving tank sounds.

4. VGA CONTROLLER

VGA controller was a component of the project that was divided into four main modules. Contained in the screen memory module are instructions on how to obtain an id that determined what glyph to get based on the coordinates of that glyph on the screen. The objective of the glyph memory module is to obtain an entire row of pixels for a specific glyph. A VGA timer circuit had a pair of counters to count horizontal and vertical ticks of the VGA clock. VGA controller circuit handled sending the screen addresses of glyph characters located on the screen to a screen memory module, computed a glyph address based on what type of glyph and what row of the glyph needed to be displayed, and displayed a specific glyph on the screen in a particular color. Below is a diagram showing how all four modules interact with one another:

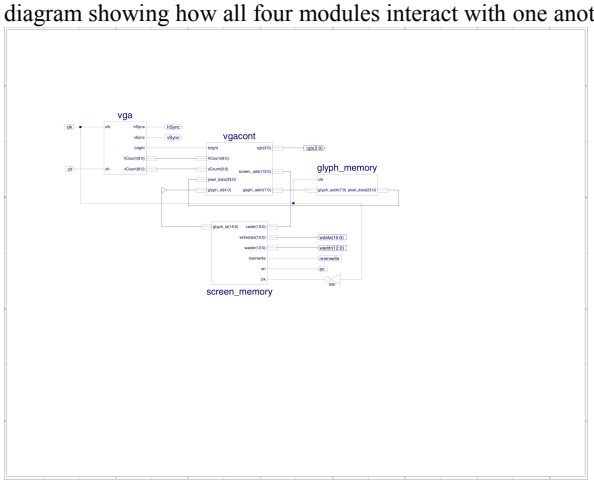

Figure 2. VGA Controller

4.1 Screen Memory

A separate block RAM was used for screen memory in order to provide access from the processor to write data and allow reading of information from the VGA controller. Since the processor uses 16 bit words, we decided to make the screen memory's width to be 16 bits as well. We set the RAM address bits to 13 bits because if we use 8x8 glyphs on a 640x480 resolution display, then we need to have 4800 8x8 glyphs to fill up the screen and having 13 bits allow for that many glyphs and more. In this module, the inputs are a clock, an enable signal, a memory write, a read address, a write address, and a write data. Screen Memory will be outputting glyph_id that determines what glyph we need at a given coordinate on the screen. A register called screen ram is used to read our glyph memory .dat file from memory. At every positive edge of the clock, if both the enable switch is on as well as the memory write switch, then screen ram will be set to the contents of write data and our glyph id will be set to the screen ram at a specific read address.

4.2 Glyph Memory

Glyphs are stored into read only block RAM. Inside the glyph memory module we have the width set to be 24 bits because we are using a 3-bit color scheme for each pixel and we have 8 pixels per row of a glyph. RAM address is set to 8 bits in order to accommodate 32 different glyphs each of which are 8 rows long. Inputs to glyph memory include a clock and a glyph address. A glyph address determines which row of a 8x8 glyph we are in so that we can display the correct part of the glyph. Outputs for our glyph module is pixel data which is an 8 pixel row of a glyph . Register glyph ram is used to read information from the glyph memory .dat file. At every positive clock edge the glyph ram will get a glyph address and store that data into pixel data which will then be outputted to the VGA controller circuit.

4.3 VGA Timer Circuit

The basics of the VGA timer circuit is to have a pair of counters used to count horizontal ticks and vertical ticks of the VGA clock. The horizontal counter will count up in 25 MHz ticks and the vertical counter will count once for each horizontal line. Inputs to the VGA timer circuit include having a system clock and clear signal. What the timer circuit will be outputting are horizontal and vertical sync signals to the VGA interface, a bright signal that is asserted whenever a pixel is bright, and a horizontal and vertical count that will tell you where you are on the screen. Registers include an enable for horizontal, an enable for vertical, a delay time parameter, and a count. VGA timer circuit has three main parts which include a clock divider, a horizontal counter, and a vertical counter.

Clock divider operates at every positive edge of the clock by setting the enable for horizontal equal to zero; if a clear signal is asserted, then count is also equal to zero, but if count is asserted, then count is reset to zero and enable horizontal gets set to one; if neither clear or count is enabled, then count will get incremented by one.

Horizontal counter begins at every positive edge of the clock by first setting the enable vertical to zero. If a clear signal is pressed, then horizontal count is set to zero. If enable horizontal is asserted, then the horizontal count will get incremented by 1. If horizontal count reaches 800 then the horizontal count will be set to zero while the enable vertical will be set to 1.

At every positive edge of the clock, if a clear signal is asserted, then the vertical count will be reset to zero. Otherwise, the vertical count will be incremented by 1 and once the vertical count reaches 521 then it will be reset to zero. If there are any changes

to the circuit then the horizontal and vertical sync will be both set to 1 and the bright signal will be set to zero. If horizontal count reaches 96 or vertical count is 2 then both horizontal and vertical sync will be set to zero. If the horizontal count is in the horizontal range of the screen between 144 and 784 and the vertical count is in the vertical range of the screen between 31 and 511 then we can assert the bright signal to display red, green, and blue values.

4.4 VGA Controller Circuit

To put it simply, the job of the VGA controller circuit is to obtain what pixels to display on the screen. There is about two and a half clock cycles between each pixel being displayed so we are utilizing a single buffer and looking up pixel by pixel.

Bright, horizontal count, vertical count, glyph id, and pixel data were the inputs to VGA controller module. VGA controller outputs an red, green, blue value as well as both a screen address and a glyph address. Registers used include display which is the two buffers that are used to store glyph data, display rgb which is set to whatever 3 bits are needed to be displayed from the buffer, and both horizontal position and vertical position. Anytime there is a change in the circuit, horizontal position is set to the horizontal count minus an offset of 144, vertical position is set to the vertical count minus an offset of 31, screen address is set to 80 times the vertical position's most significant 6 bits added with horizontal position's most significant 7 bits, and glyph address is a concatenation of glyph id and vertical position's lower 3 bits. The buffers contained in display will be set to the pixel data that is outputted from the glyph memory. There are 8 different cases for each row of an 8x8 glyph which basically set the display rgb to whatever 3 bits the buffer needs to display. If bright is asserted, then we will set our output red, green, and blue to be what the display rgb is set to which will give us a color for the pixel displayed.

5. GAME CODE

5.1 Assembly Code Converter

In order to more effectively convert assembly code into hexadecimal numbers which could be read by the processor, a Program, coded in C-sharp, was developed for the specific purpose of encoding the correct hexadecimal digits from the CR-16 assembly code instructions. This program encoded each instruction as a line of 4 hexadecimal digits which would then be written to a .dat file that could be directly used by the processor. One feature of this 'assembler' is the ability to read labels used in both branch and jump instructions as opposed to numerical offsets or PC locations stored in registers. For the case of reading in labels, jump instructions would add lines of hexadecimal code to the final .dat file which in turn would put the PC location of the appropriate label into a register.

5.2 Tank Wars Assembly Code

The program itself was broken into sections. Most of the code is contained in Functions which is called with the Jump and Link assembly instruction. All of the functions are organized at the end of the code with the program code at the beginning of the file. The functions takes care of things such as displaying a tank, clearing a tank, moving a tank, displaying a splash screen, as well as moving and firing bullets from the tanks. The program code is basically a beginning section which initializes a splash screen and waits for an input, such as a press start prompt, from a player's controller; a main program loop, running continuously until a tank is hit, that displays tank and bullet movement and checks for a collision between bullets and tanks; the final part of the program displays a game over message as well as the message stating which player won the game. After that, the game waits for input from the player to reset the game.

6. NES CONTROLLERS

The inputs used for our game are two NES controllers. NES controllers uses an internal parallel to serial shift register (CMOS 4021) that latches the button states before shifting them out one at a time to the system. Pinout for the NES controller is shown below:

Table 1. NES controller pinout

Pin #	Function	Wire
1	Ground	Brown
2	Clock	Red
3	Latch	Orange
4	Data Out	Yellow
5	No Connect	None
6	No Connect	None
7	Power	White

A timing diagram that shows the input buttons shifting out in a sequence is listed below:

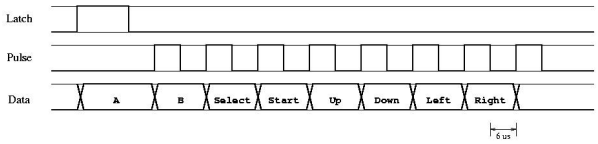

Figure 3. Button input timing diagram

A, the first button input, was obtained at the fallen edge of the latch while the rest of the button inputs were obtained at the fallen edge pulses. In order to use the two NES controllers for the Spartan-3E FPGA board, we cut the cables on the NES controllers and soldered the wires onto two clips which were hooked up to the two extensions on the Spartan-3E FPGA board.

7. SOUND

We opted to create the sound effects used in the game manually instead of reading a pre-generated sound file. The sounds generated were purely dependent on a sequence of notes. Using a combination of frequency and time, we were able to make each note. Frequency determines what sound the note will play and time determines the duration of a note. Utilizing the on-board 50 MHz clock on the Spartan-3E FPGA board, we were able to generate frequencies. These frequencies are sent out to an amp which is connected to a speaker to produce an audible sound. The frequency range from 100 Hz to 20 kHz is what humans can normally detect. Randomly choosing the frequency we need for

our sound can take up a lot of time so we just used some example .wav files and graphed those in Matlab. The fourier transform was used to obtain the frequencies of each note. Below is an example of the first six notes of Beethoven's classic “Fifth Symphony” in a .wav file:

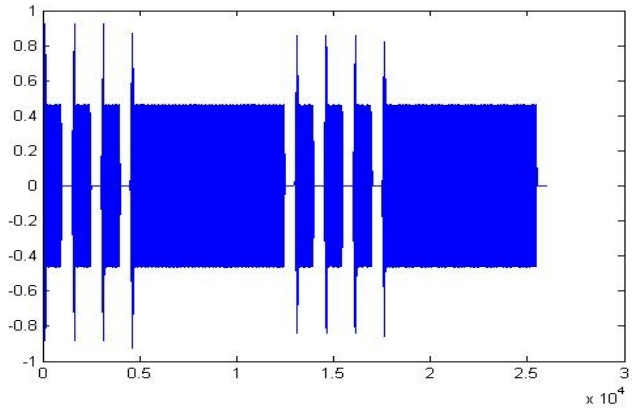

Figure 4. Fifth Symphony example

Take the fourier transform of a note will show the frequency of the note followed by some harmonics. The following diagram shows magnitude vs. frequency for notes 1-3 because they are the same notes:

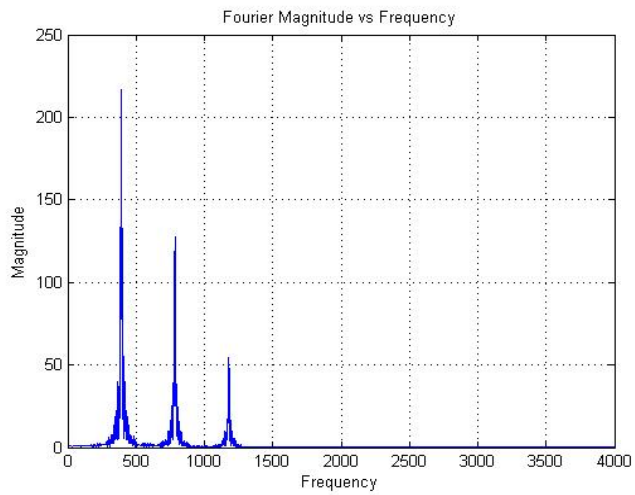

Figure 5. Magnitude vs. frequency graph

In our project, we are using four unique sounds for game start, game over, bullet, and moving a tank. Game start and game over sounds are designed to introduce and end our game. Sound effects were created using the bullet and tank moving sounds. Analyzing how other sound waves looked like in Matlab helped us to generate our own unique sound. In generating our game start sound, we took the original Battle City's game start sound, analyzed it in Matlab, and recorded all the frequencies at each note to see the range of the frequencies, where pauses and durations are, and how the overall sound was organized. Different frequencies were used on the notes and there were increases and decreases on pauses and durations at various places. Utilizing the above method, we were able to generate all of our sounds.

8. RESULTS

The result of our project is a functional 2D tank shooter based video game that runs on the Spartan-3E FPGA board. It may be lacking in features, but it is playable. The game begins with two tanks that begin on opposite sides of the screen and the objective of the game is to destroy the opposing tank using one mine. Once you use up a mine, you have no more ammunition to use to destroy the opponent so you must lure your enemy into stepping on your mine. The mine you lay is friendly to you so you can run through your own mine without penalty. The game ends when a tank steps on an opposing tank's mine or is shot point blank with a bullet from the tank. Originally we planned for moving bullets that the tank fires, but in the current state the bullets will not move. The CPU, VGA controller, and NES controller are all working and testing has confirmed this. ISE simulation testing was our tool for debugging our modules except for the sound. The sound was tested by uploading the code for sound onto the Spartan-3E FPGA board. An amplifier was connected to the FPGA board as well as a speaker to determine what sound was produced. Each module was tested individually and then all modules were combined into a final project and tested as one:

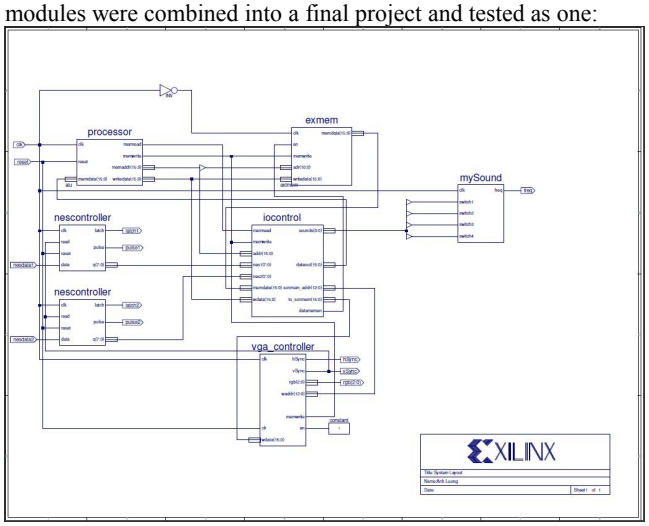

Figure 6. Final project schematic

9. CONCLUSION

Utilizing both Verilog HDL and assembly code, we were able to create a video game called Tank Wars. Tank Wars runs on the Spartan-3E FPGA board and features two player simultaneous play where each player controls a tank and tries to destroy the opposing player's tank. Our video game consists of six major parts which are the Processor, Memory Map I/O, VGA controller, Game Code, NES controllers, and Sound. The processor will fetch, decode, and execute instructions. Memory Map I/O determines where to map information to memory such as game code, program data, screen memory, glyph memory, NES controller button inputs, and sounds. VGA controller retrieves glyph data from memory and outputs the colored pixels on a display. Game Code decides how tanks interact with each other in our video game. NES controllers provide an input for controlling tanks in the game. Sound provides users with audio for when the game begins and ends as well as sound effects for tank bullets and when a tank moves.

What we did well in this project is having a working, running video game. Although our game is functional, it is very bare bones and has much room to improve upon. Some improvements include having more detailed tanks as well as having bullets that move across the screen. In addition, our Tank Wars video game lacks some features that are included in Battle City such as bases for tanks, more enemy tanks, and obstacles such as trees and water. Furthermore, a nice addition for the sound would have been adding another speaker which would allow simultaneous play of two sounds at once. With an additional speaker, we could have added an in-game soundtrack with sound effects or added harmonics with the sounds.

10. ACKNOWLEDGMENTS

Our thanks to Professor Erik Brunvand and the teaching assistant Michael Kingston for providing us with various diagrams, links, notes, and user guides which aided us tremendously in constructing the various components of our project.

11. REFERENCES

[1] CS/EE 3710 RISC Processor. Retrieved November 2, 2010, from University of Utah: http://eng.utah.edu/~cs3710/handouts/ISA.pdf

[2] Pinouts.RU: Nintendo NES and SNES joystick controller connector pinout, 2008. Retrieved November 2, 2010, from: http://pinouts.ru/Game/snescontroller_pinout.shtml

[3] Spartan-3 Generation FPGA User Guide. Retrieved November 2, 2010, from University of Utah: http://eng.utah.edu/~cs3710/xilinx-docs/ug331.pdf

[4] Spartan-3E FPGA Family: Complete Data Sheet. Retrieved November 2, 2010, from University of Utah: http://eng.utah.edu/~cs3710/xilinx-docs/ds312.pdf

[5] Spartan-3E FPGA Starter Kit Board User Guide. Retrieved November 2, 2010, from University of Utah: http://eng.utah.edu/~cs3710/xilinx-docs/ug230.pdf

[6] Using Block RAM in Spartan-3 Generation FPGAs. Retrieved November 2, 2010, from University of Utah: http://eng.utah.edu/~cs3710/xilinx-docs/xapp463.pdf